

OFFLINE REINFORCEMENT LEARNING WITH IMPLICIT Q-LEARNING

Presenter: Hongming Zhang

Date: 2023.03.29

Introduction



Most current offline reinforcement learning methods need to query the value of unseen actions during training. This paper proposes a new offline RL method that [never needs to query unseen actions](#):

- Treat the state value function as a random variable related to action
- Estimate the best value of a state with [upper expectile](#)
- Fit the upper expectile value function and back it up into a [Q-function](#)
- Extract policy via [advantage-weighted behavioral cloning](#) from Q-function

Motivation



Two conflicting aims in offline reinforcement learning:

- learn a policy that **improves** over the behavior policy
- **Stay close** to behavior policy to avoid **extrapolation errors** due to distributional shift

Can we devise an offline RL method that avoids extrapolation errors by **never querying or estimating values for actions that were not seen** in the data?

- Approximate an upper expectile of the distribution over values with respect to **the distribution of dataset actions** for each state
- Alternate between fitting this value function with expectile regression, and then using it to compute Bellman backups for training the Q-function

Algorithm



Q-learning:

$$L_{TD}(\theta) = \mathbb{E}_{(s,a,s') \sim \mathcal{D}} [L_2(r(s,a) + \gamma \max_{a'} Q_{\hat{\theta}}(s', a') - Q_{\theta}(s, a))]$$

SARSA:

$$L(\theta) = \mathbb{E}_{(s,a,s',a') \sim \mathcal{D}} [L_2(r(s,a) + \gamma Q_{\hat{\theta}}(s', a') - Q_{\theta}(s, a))]$$

where $L_2(u) = u^2$, $\hat{\theta}$ is target network

	Pro	Con
Q learning:	Q^*	extrapolation errors in offline case
SARSA:	in-distribution sample	$Q^{\pi_{\beta}}$
Our objective - Q^* in offline case		

Algorithm



Q-learning:

$$L_{TD}(\theta) = \mathbb{E}_{(s,a,s') \sim \mathcal{D}} [L_2(r(s,a) + \gamma \max_{a'} Q_{\hat{\theta}}(s', a') - Q_{\theta}(s, a))]$$

Pro: Q^*

Con: extrapolation
errors/distributional shift

SARSA:

$$L(\theta) = \mathbb{E}_{(s,a,s',a') \sim \mathcal{D}} [L_2(r(s,a) + \gamma Q_{\hat{\theta}}(s', a') - Q_{\theta}(s, a))]$$

Pro: in-distribution sample

Con: $Q^{\pi_{\beta}}$

Our objective:

$$L(\theta) = \mathbb{E}_{(s,a,s') \sim \mathcal{D}} [L_2(r(s,a) + \gamma \max_{a' \in \mathcal{A}, \pi_{\beta}(a'|s') > 0} Q_{\hat{\theta}}(s', a') - Q_{\theta}(s, a))]$$

where $L_2(u) = u^2$, $\hat{\theta}$ is target network

How to obtain π_{β} ? / How to compute $r(s,a) + \gamma \max_{a' \in \mathcal{A}, \pi_{\beta}(a'|s') > 0} Q_{\hat{\theta}}(s', a')$?

Algorithm



Least squares regression:

$$\arg \min_m \mathbb{E}_{x \sim X} [L_2(x - m)] \text{ where } L_2(u) = u^2$$

Expectile regression: asymmetric least squares regression:

$$\arg \min_{m_\tau} \mathbb{E}_{x \sim X} [L_2^\tau(x - m_\tau)] \text{ where } L_2^\tau(u) = |\tau - \mathbb{I}(u < 0)|u^2$$

Algorithm

Expectile regression: asymmetric least squares regression:

$$\arg \min_{m_\tau} \mathbb{E}_{x \sim X} [L_2^\tau(x - m_\tau)] \text{ where } L_2^\tau(u) = |\tau - \mathbb{I}(u < 0)|u^2$$

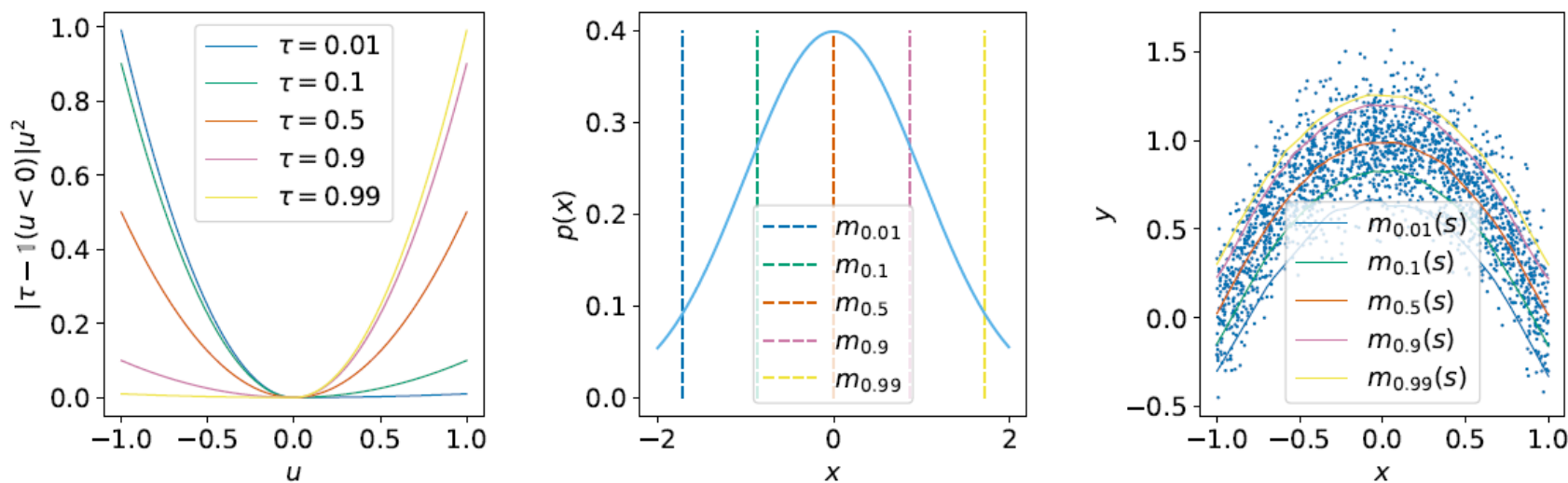


Figure 1: **Left:** The asymmetric squared loss used for expectile regression. $\tau = 0.5$ corresponds to the standard mean squared error loss, while $\tau = 0.9$ gives more weight to positives differences. **Center:** Expectiles of a normal distribution. **Right:** an example of estimating state conditional expectiles of a two-dimensional random variable. Each x corresponds to a distribution over y . We can approximate a maximum of this random variable with expectile regression: $\tau = 0.5$ correspond to the conditional mean statistics of the distribution, while $\tau \approx 1$ approximates the maximum operator over in-support values of y .

Algorithm



Use expectile regression to approximate $r(s, a) + \gamma \max_{a' \in \mathcal{A}, \pi_\beta(a'|s') > 0} Q_{\hat{\theta}}(s', a')$:

$$L(\theta) = \mathbb{E}_{(s,a,s',a') \sim \mathcal{D}} [L_2^\tau(r(s, a) + \gamma Q_{\hat{\theta}}(s', a') - Q_\theta(s, a))],$$

where $L_2^\tau(u) = |\tau - \mathbb{I}(u < 0)|u^2$

This also defines the entire spectrum of methods between **SARSA** ($\tau = 0.5$) and **Q-learning** ($\tau \rightarrow 1$).

Disentangle this formulation into two parts to make the learning more stable:

$$L_V(\psi) = \mathbb{E}_{(s,a) \sim \mathcal{D}} [L_2^\tau(Q_{\hat{\theta}}(s, a) - V_\psi(s))]$$

$$L_Q(\theta) = \mathbb{E}_{(s,a,s') \sim \mathcal{D}} [L_2(r(s, a) + \gamma V_\psi(s') - Q_\theta(s, a))]$$

Algorithm

Extract the policy using advantage weighted regression:

$$L_{\pi}(\phi) = \mathbb{E}_{(s,a) \sim \mathcal{D}} [\exp \left(\beta \left(Q_{\hat{\theta}}(s, a) - V_{\psi}(s) \right) \right) \log \pi_{\phi}(a|s)]$$

where $\beta \in [0, \infty)$ is an inverse temperature.

Nair, Ashvin, et al. "Awac: Accelerating online reinforcement learning with offline datasets." arXiv preprint arXiv:2006.09359 (2020).

Algorithm 1 Implicit Q-learning

Initialize parameters $\psi, \theta, \hat{\theta}, \phi$.

TD learning (IQL):

for each gradient step **do**

$\psi \leftarrow \psi - \lambda_V \nabla_{\psi} L_V(\psi)$

$\theta \leftarrow \theta - \lambda_Q \nabla_{\theta} L_Q(\theta)$

$\hat{\theta} \leftarrow (1 - \alpha) \hat{\theta} + \alpha \theta$

end for

Policy extraction (AWR):

for each gradient step **do**

$\phi \leftarrow \phi - \lambda_{\pi} \nabla_{\phi} L_{\pi}(\phi)$

end for

Experiments

Maze environment

The dataset consists of 1 optimal trajectory and 99 trajectories with uniform random actions.

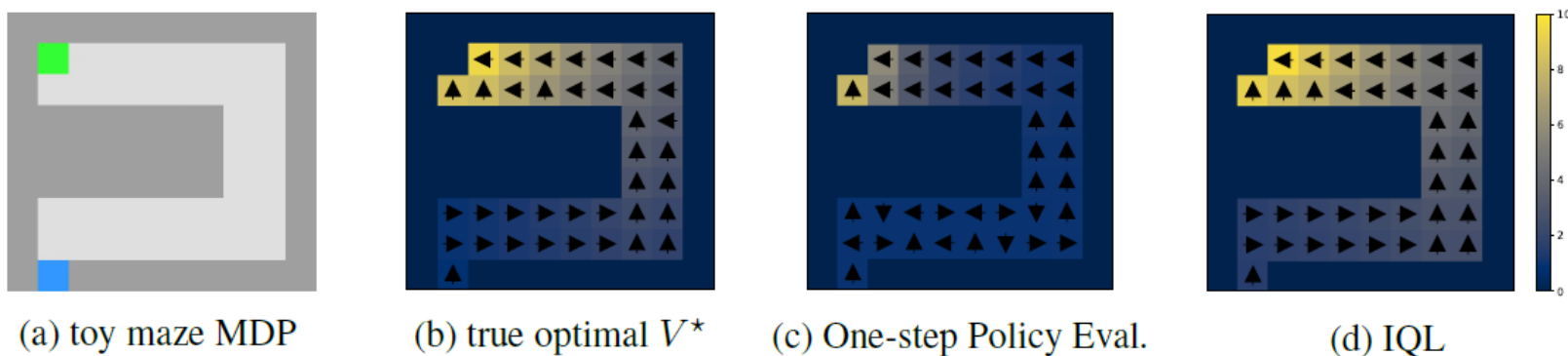


Figure 2: Evaluation of our algorithm on a toy umaze environment (a). When the static dataset is heavily corrupted by suboptimal actions, one-step policy evaluation results in a value function that degrades to zero far from the rewarding states too quickly (c). Our algorithm aims to learn a near-optimal value function, combining the best properties of SARSA-style evaluation with the ability to perform multi-step dynamic programming, leading to value functions that are much closer to optimality (shown in (b)) and producing a much better policy (d).

Experiments

Comparisons on offline RL benchmarks

Table 1: Averaged normalized scores on MuJoCo locomotion and Ant Maze tasks. Our method outperforms prior methods on the challenging Ant Maze tasks, which require dynamic programming, and is competitive with the best prior methods on the locomotion tasks.

Dataset	BC	10%BC	DT	AWAC	Onestep RL	TD3+BC	CQL	IQL (Ours)
halfcheetah-medium-v2	42.6	42.5	42.6	43.5	48.4	48.3	44.0	47.4
hopper-medium-v2	52.9	56.9	67.6	57.0	59.6	59.3	58.5	66.3
walker2d-medium-v2	75.3	75.0	74.0	72.4	81.8	83.7	72.5	78.3
halfcheetah-medium-replay-v2	36.6	40.6	36.6	40.5	38.1	44.6	45.5	44.2
hopper-medium-replay-v2	18.1	75.9	82.7	37.2	97.5	60.9	95.0	94.7
walker2d-medium-replay-v2	26.0	62.5	66.6	27.0	49.5	81.8	77.2	73.9
halfcheetah-medium-expert-v2	55.2	92.9	86.8	42.8	93.4	90.7	91.6	86.7
hopper-medium-expert-v2	52.5	110.9	107.6	55.8	103.3	98.0	105.4	91.5
walker2d-medium-expert-v2	107.5	109.0	108.1	74.5	113.0	110.1	108.8	109.6
locomotion-v2 total	466.7	666.2	672.6	450.7	684.6	677.4	698.5	692.4
antmaze-umaze-v0	54.6	62.8	59.2	56.7	64.3	78.6	74.0	87.5
antmaze-umaze-diverse-v0	45.6	50.2	53.0	49.3	60.7	71.4	84.0	62.2
antmaze-medium-play-v0	0.0	5.4	0.0	0.0	0.3	10.6	61.2	71.2
antmaze-medium-diverse-v0	0.0	9.8	0.0	0.7	0.0	3.0	53.7	70.0
antmaze-large-play-v0	0.0	0.0	0.0	0.0	0.0	0.2	15.8	39.6
antmaze-large-diverse-v0	0.0	6.0	0.0	1.0	0.0	0.0	14.9	47.5
antmaze-v0 total	100.2	134.2	112.2	107.7	125.3	163.8	303.6	378.0
total	566.9	800.4	784.8	558.4	809.9	841.2	1002.1	1070.4
kitchen-v0 total	154.5	-	-	-	-	-	144.6	159.8
adroit-v0 total	104.5	-	-	-	-	-	93.6	118.1
total+kitchen+adroit	825.9	-	-	-	-	-	1240.3	1348.3
runtime	10m	10m	960m	20m	$\approx 20m^*$	20m	80m	20m

*: Note that it is challenging to compare one-step and multi-step methods directly. Also, [Brandfonbrener et al. \(2021\)](#) reports results for a set of hyperparameters, such as batch and network size, that is significantly different from other methods. We report results for the original hyperparameters and runtime for a comparable set of hyperparameters.

Experiments

Online fine-tuning after offline RL

Dataset	AWAC	CQL	IQL (Ours)
antmaze-umaze-v0	56.7 → 59.0	70.1 → 99.4	86.7 → 96.0
antmaze-umaze-diverse-v0	49.3 → 49.0	31.1 → 99.4	75.0 → 84.0
antmaze-medium-play-v0	0.0 → 0.0	23.0 → 0.0	72.0 → 95.0
antmaze-medium-diverse-v0	0.7 → 0.3	23.0 → 32.3	68.3 → 92.0
antmaze-large-play-v0	0.0 → 0.0	1.0 → 0.0	25.5 → 46.0
antmaze-large-diverse-v0	1.0 → 0.0	1.0 → 0.0	42.6 → 60.7
antmaze-v0 total	107.7 → 108.3	151.5 → 231.1	370.1 → 473.7
pen-binary-v0	44.6 → 70.3	31.2 → 9.9	37.4 → 60.7
door-binary-v0	1.3 → 30.1	0.2 → 0.0	0.7 → 32.3
relocate-binary-v0	0.8 → 2.7	0.1 → 0.0	0.0 → 31.0
hand-v0 total	46.7 → 103.1	31.5 → 9.9	38.1 → 124.0
total	154.4 → 211.4	182.8 → 241.0	408.2 → 597.7

Table 2: Online finetuning results showing the initial performance after offline RL, and performance after 1M steps of on-line RL. In all tasks, IQL is able to finetune to a significantly higher performance than the offline initialization, with final performance that is comparable to or better than the best of either AWAC or CQL on all tasks except pen-binary-v0.

Conclusion



This paper presented implicit Q-Learning (IQL), a general algorithm for offline RL that completely [avoids any queries to values of out-of-sample actions](#) during training while still enabling multi-step dynamic programming.

- Computationally efficient
- Simple to implement
- Excellent performance



THANKS!